

Maritime CargoService: Sprint 0

Davide Chirichella, Gabriele Doti, Daniele
Maccagnan

Ingegneria dei Sistemi Software, A.A. 2025/2026
Alma Mater Studiorum . Università di Bologna

June 23, 2026

1. Introduction

Una compagnia di trasporto marittimo di container (d'ora in poi *la committente*) intende automatizzare le operazioni di carico dei container nella stiva della nave (d'ora in poi **hold**). A tal fine prevede di impiegare un robot a guida differenziale (Differential Drive Robot, d'ora in poi **cargorobot**).

L'obiettivo dello Sprint 0 è formalizzare i requisiti forniti dalla committente in modo preciso e non ambiguo, costruire un primo modello logico dei macro-componenti del sistema, evidenziare il *core business*, motivare la scelta del linguaggio di modellazione e definire un primo insieme di piani di test funzionali.

Ogni scelta è strettamente ancorata ai requisiti: il modello qui presentato serve a catturare il comportamento richiesto, senza anticipare decisioni progettuali o scelte implementative che verranno affrontate negli sprint successivi.

La traccia del progetto può essere scaricata dal seguente [link](#)^o

2. Requirements

L'azienda richiede di realizzare un servizio denominato **cargoservice** con il seguente funzionamento:

- Il cargoservice è in grado di ricevere una richiesta di carico di un container inviata da un cliente tramite il pulsante dell'IOPort.
- Invia la risposta **retrylater** se l'IOPort è attualmente occupato da un container oppure se il sistema è **Out of service**.
- Rifiuta la richiesta quando la hold è già piena, ovvero gli slot1-4 sono già tutti occupati.
- Altrimenti, considera il sistema come **engaged**, rileva uno slot libero e restituisce come risposta il nome dello slot riservato. Mentre il sistema è engaged, il LED deve lampeggiare.
- Quando la richiesta di carico viene accettata, il cliente deve spostare il container nell'area del sensore entro un tempo prefissato (ad esempio 30 secondi), altrimenti il sistema diventa **disengaged**.
- Successivamente, il cargoservice utilizza il cargorobot per spostare il container dall'IOPort a slot5 (per l'etichettatura del container) e poi allo slot riservato.
- Il servizio deve inoltre mostrare sul display dell'IOPort:
 - lo stato attuale della hold
 - il messaggio "**Service working**" quando tutto sta procedendo correttamente
 - il messaggio "**Out of service**" se il sensore sonar misura una distanza $D > D_{FREE}$ per almeno 3 secondi (possibile guasto del sonar)

2.1. Domande aperte alla committente

Domanda al committente.

Posizione dell'IOPort nella mappa. I requisiti indicano che il cargorobot sposta

il container *dall'IOPort a slot5*, lasciando intendere che l'IOPort sia una cella praticabile. Come si colloca nella griglia?

Domanda al committente.

Richieste concorrenti. Il sistema deve bufferizzare più richieste di carico contemporanee, oppure una nuova richiesta ricevuta mentre il sistema è *engaged* viene semplicemente rifiutata / respinta con *retrylater* senza accodamento? Dai requisiti si interpreta che le richieste **non siano bufferizzate**: se il sistema è occupato, la risposta è immediata (*retrylater* o *refused*) senza code di attesa.

Domanda al committente.

Liberazione degli slot e aggiornamento della disponibilità. I requisiti descrivono il processo di carico dei container negli slot1-4, ma non specificano come venga gestita la liberazione di uno slot già occupato. Possiamo assumere che lo svuotamento degli slot sia effettuato da sistemi o operatori esterni al nostro sistema? In tal caso, con quale meccanismo viene notificato a cargoservice che uno specifico slot è stato liberato e può tornare disponibile per future prenotazioni?

Domanda al committente.

Ripristino dello stato Service working. I requisiti specificano che il sistema entra nello stato *Out of service* quando il sonar rileva una distanza $D > D_{\text{FREE}}$ per almeno 3 secondi. Non è invece specificata la condizione per il ritorno allo stato *Service working*. Il sistema deve tornare operativo non appena il sonar rileva $D \leq D_{\text{FREE}}$ oppure è richiesto un ulteriore intervallo di stabilità prima del ripristino del servizio?

3. Requirement analysis

3.1. Core business

Il **core business** del sistema è la gestione del ciclo di carico di un container. La responsabilità della sequenza applicativa resta in **cargoservice**: il cargorobot è coinvolto per eseguire spostamenti richiesti dal servizio, non per decidere se una richiesta debba essere accettata, rifiutata o sospesa. La sequenza principale ricavata dai requisiti è:

1. Il customer preme il pushbutton dell'IOPort.
2. cargoservice verifica le precondizioni (stato sistema, IOPort, disponibilità slot).
3. Se soddisfatte: stato *engaged*, prenotazione slot, notifica al customer.
4. Il customer deposita il container nell'area del sonar entro il timeout.
5. cargoservice comanda al cargorobot di spostare il container da IOPort a slot5.
6. Il marker device etichetta il container e segnala il completamento.
7. cargoservice comanda al cargorobot di spostare il container da slot5 allo slot riservato; il sistema torna *disengaged*.

```

System cargosystem

Request load_request : load_request(X)
Reply load_accepted : load_accepted(SLOT) for load_request
Reply load_refused : load_refused(REASON) for load_request
Reply load_retrylater : load_retrylater(REASON) for load_request

// Evento per simulare la percezione del container da parte del sonar
// (D < DFREE/2 per ≥ 3 secondi)
Event container_detected : container_detected(D)

Context ctxcargoservice ip [host="localhost" port=8050]

QActor cargoservice context ctxcargoservice {

    State s0 initial {
        println("cargoservice | started")
    }
    Goto disengaged

    // STATO: DISENGAGED (Il sistema è pronto ad accogliere richieste)
    State disengaged {
        println("cargoservice | DISENGAGED: waiting for load_request...")
    }

    Transition t0
        whenRequest load_request → handle_load_request

    // STATO: VALUTAZIONE PRECONDIZIONI
    State handle_load_request {
        replyTo load_request with load_accepted : load_accepted(1)
    }
    Goto engaged // Assumendo che la richiesta sia accettata

    // STATO: ENGAGED (Il sistema attende l'azione fisica del customer)
    State engaged {
        println("cargoservice | ENGAGED: waiting for customer to deposit
container...")
    }
    Transition t1
        whenTime 10000 → handle_timeout // Requisito: Timeout
deposito
        whenEvent container_detected → move_to_slot5 // Requisito:
Rilevazione container

    // STATO: GESTIONE TIMEOUT (Il customer non ha depositato in tempo)
    State handle_timeout {
        println("cargoservice | TIMEOUT: customer did not deposit in time.
Freeing IOPort...")
        // Logica per liberare la prenotazione
    }
    Goto disengaged

    // STATO: PRESA IN CARICO DEL CONTAINER
    State move_to_slot5 {

```

```

        println("cargoservice | CONTAINER RILEVATO: avvio movimentazione
verso slot5...")
        // NOTA ANALISI: In Sprint successivi qui ci sarà l'interazione
        (Dispatch/Request)
        // con il 'cargorobot' per lo spostamento fisico.
    }
    Goto disengaged // Al termine del ciclo di carico, torna disponibile
}

```

3.2. Macro-componenti e natura software

COMPONENTE	RUOLO	STATO
cargoservice	Orchestratore principale. Natura reattiva (richieste in arrivo) e proattiva (comandi al cargorobot, aggiornamenti display). Non può restare privo di controllo: da requisito deve rispondere e agire autonomamente.	Da sviluppare
cargorobot	Entità che governa il DDR per la movimentazione fisica richiesta dal cargoservice. La scelta POJO vs microservizio è rimandata all'analisi: se fosse un POJO, il cargoservice gli cederebbe il controllo (trasferimento di controllo sincrono) e non potrebbe reagire ad altri eventi durante la movimentazione. Se invece fosse un microservizio, la comunicazione sarebbe asincrona e message-driven, con un disaccoppiamento molto maggiore. Forti motivazioni spingono verso la seconda opzione, ma la decisione è oggetto degli sprint successivi.	Da chiarire / da sviluppare
IOPort	Interfaccia con il customer (pushbutton + dis-	Dispositivo fisico fornito sw da sviluppare

	play). Se realizzata come microservizio separato (rappresentata per esempio tramite una Single Page Application (SPA), il pushbutton come un button HTML e il display come textarea HTML), le responsabilità sono completamente disaccoppiate da cargoservice e i due possono essere sviluppati in parallelo; l'unica interfaccia condivisa è il messaggio <i>load_request</i> definito in questo sprint.	
sonar	Sensore di distanza.	Dispositivo fisico fornito driver sw da sviluppare
marker device	Etichettatura in slot5.	Dispositivo fisico simulato fornito sw da sviluppare
LED	Indicatore stato <i>engaged</i> . Può essere fisico o virtuale (rappresentato ad esempio come un indicatore visivo lampeggiante, eventualmente nella stessa SPA dell'IOPort). La scelta dipende dall'infrastruttura disponibile.	Fisico o virtuale sw da sviluppare
hold	Struttura dati per lo stato della stiva (occupazione slot). Passivo.	Da sviluppare

3.3. Motivazione dell'uso del linguaggio QAK

QAK è il linguaggio messo a disposizione dalla nostra software house (unibo.issLab) per modellare sistemi software distribuiti, particolarmente espressivo nel formalizzare il concetto di **attore autonomo** e di **messaggio**, riducendo di molto l'“Abstraction Gap” tra requisiti e modello. La natura **reattiva e proattiva** di cargoservice, che deve rispondere a stimoli esterni e avviare autonomamente sequenze di azioni, è infatti catturata in modo naturale da un attore QAK, cosa che un POJO (Plain Old Java Object), componente passivo attivato da chiamate sincrone, non catturerebbe altrettanto bene. Dal modello QAK viene generato automaticamente codice Kotlin eseguibile,

il che consente di disporre di un **primo prototipo osservabile** già nello **Sprint 0**, prima ancora di scrivere una riga di logica applicativa.

3.4. Formalizzazione dei messaggi QAK

La seguente formalizzazione non introduce ancora scelte di progetto, limitandosi ad elencare i messaggi necessari per esprimere i requisiti. **Request/Reply** viene usato quando il requisito prevede una risposta osservabile; **Dispatch** quando il requisito parla di una segnalazione o di un aggiornamento senza risposta diretta.

3.4.1. customer / IOPort -> cargoservice

Request	load_request	:	loadRequest(none)	
Reply	load_accepted	:	loadAccepted(slotID)	for load_request
Reply	load_retrylater	:	loadRetryLater(none)	for load_request
Reply	load_refused	:	loadRefused(none)	for load_request

La Request `load_request` rappresenta il momento nel quale viene effettuata una richiesta tramite il pushbutton dell'IOPort e le 3 Reply sono le relative risposte del cargoservice: nel caso la richiesta venisse accettata si consegna anche l'ID dello slot nel quale va posizionato il carico.

3.4.2. sonar -> cargoservice

Dispatch	sonar_data	:	sonarData(distance)
Dispatch	container_detected	:	containerDetected(none)
Dispatch	sonar_failure	:	sonarFailure(none)

Il Dispatch `sonar_data` rappresenta il monitoraggio costante del Sensore della distanza tra il container e la sensor_area. Quando viene rilevato un container entro la distanza $D > D_{\text{FREE}}$ per ≥ 3 s verrà inviato il Dispatch `container_detected`, mentre se la distanza è $D < D_{\text{FREE}}/2$ per ≥ 3 s verrà inviato `sonar_failure`.

Nota: La scelta **Dispatch** (e non **Request**) per i messaggi del sonar è una **ipotesi di analisi**: il sonar segnala eventi al cargoservice senza aspettarsi una risposta diretta. Se l'analisi rivelasse la necessità di una conferma (es. handshake di rilevazione), il tipo del messaggio andrà rivisto nello sprint 1.

3.4.3. cargoservice -> cargorobot

Dispatch	move_to_slot5	:	moveToSlot5(none)
Dispatch	move_to_slot	:	moveToSlot(slotID)

I Dispatch `move_to_slot5` e `move_to_slot` rappresentano rispettivamente le richieste di spostamento da parte di cargoservice al robot. Si preferisce usare dei Dispatch al posto del modello Request/Reply per il possibile lungo tempo che potrebbe intercorrere tra una Request e la sua eventuale Reply.

3.4.4. cargorobot -> cargoservice

```
Dispatch move_done      : moveDone(none)
Dispatch move_done      : moveDone(none)
```

I Dispatch `move_done` sono le corrispettive risposte a `move_to_slot5` e `move_to_slot`. La `move_to_slot5` avverrà solamente quando il container in questione dovrà essere sottoposto al marker, subito dopo cargoservice chiederà al robot di spostare quel container in uno degli altri slot.

3.4.5. marker device -> cargoservice

```
Dispatch marking_done : markingDone(containerID)
```

Messaggio di conferma a cargoservice di fine lavoro del marker.

3.4.6. cargoservice -> marker device

```
Dispatch start_marking : startMarking(containerID)
```

Messaggio di richiesta di inizio lavoro al marker, avverrà quando un container verrà depositato nello slot5.

3.4.7. cargoservice -> display / LED

```
Dispatch display_hold_state : holdState(state)
Dispatch display_status     : displayStatus(message)
Dispatch led_on              : ledOn(none)
Dispatch led_off             : ledOff(none)
```

Questi Dispatch rappresentano rispettivamente la “consegna” dei dati sullo status generale e sullo stato della hold da parte del cargoservice al display e le richieste di accendere o spegnere il LED. Le richieste al LED avverranno in corrispondenza di eventuali cambi di stato (**engaged** o **disengaged**).

3.5. Contesti logici

CONTESTO	COMPONENTI E RESPONSABILITÀ
ctxCargoService	Contiene l'attore cargoservice. Nucleo del comportamento richiesto e punto di orchestrazione del ciclo di carico.
ctxIO	Raggruppa le entità legate all'IOPort e ai dispositivi citati dai requisiti: ioport, sonar, led, markerdevice.
ctxRobot	Raggruppa le entità legate al cargorobot e alla movimentazione richiesta.

3.6. Schema della hold

Legenda: H = HOME S = sonar/sensor area 1-4 = slot1-4
5 = slot5 I = IOPort X = ostacolo . = libero

```

      col0 col1 col2 col3 col4 col5 col6
riga0 [ H ][ S ][ . ][ . ][ . ][ . ][ . ]
riga1 [ . ][ 1 ][ X ][ X ][ 2 ][ . ][ . ]
riga2 [ . ][ . ][ . ][ . ][ X ][ 5 ][ . ]
riga3 [ . ][ 3 ][ X ][ X ][ 4 ][ . ][ . ]
riga4 [ . ][ . ][ . ][ . ][ . ][ . ][ . ]
riga5 [ I ][ X ][ X ][ X ][ X ][ X ][ X ]

```

Nota: La posizione esatta dell'IOPort e del sonar richiede conferma dalla committente. Lo schema è una prima interpretazione fedele alla figura allegata ai requisiti.

4. Test plan

I test funzionali verificano il comportamento osservabile del sistema rispetto ai requisiti, indipendentemente dall'implementazione.

SCENARIO	PRECONDIZIONI	RISULTATO ATTESO
Accettazione	disengaged , non OoS, IOPort libera, slot disponibile	load_accepted con slot riservato; stato engaged ; LED lampeggiante
Rifiuto (hold piena)	disengaged , non OoS, IOPort libera, slot1-4 tutti occupati	load_refused ; stato disengaged ; LED spento
Retrylater (OoS)	Sistema Out of service	load_retrylater ; stato immutato
Retrylater (IOPort occupata)	disengaged , non OoS, IOPort occupata da un container	load_retrylater ; stato disengaged
Timeout deposito	Sistema engaged ; customer non deposita entro il tempo prefissato	Sistema disengaged ; LED spento
Ciclo completo	disengaged , non OoS, IOPort libera, slot disponibile	Container nello slot riservato; display “Service working” ; disengaged ; LED spento

Guasto sonar	Sistema operativo; sonar misura $D > D_{\text{FREE}}$ per ≥ 3 s	Sistema Out of service ; display “ Out of service ”
Rilevazione container	Sistema engaged ; sensor area vuota; sonar misura $D < D_{\text{FREE}}/2$ per ≥ 3 s	cargoservice avvia la movimentazione verso slot5

5. Project

Dall’analisi dei requisiti si propone, come ipotesi iniziale, una struttura a **tre sprint** con integrazione progressiva dei componenti. La suddivisione serve a organizzare il lavoro e i test, non a fissare decisioni architetturali definitive.

5.1. Sprint 1: Core business

Goal: realizzare il primo nucleo eseguibile di cargoservice con collaboratori simulati.

Funzionalità: ciclo di carico completo con collaboratori simulati, modello dello stato della hold, LED simulato, timeout.

5.2. Sprint 2: IOPort, display e sonar

Goal: rendere esplicita l’interazione con IOPort, display e sonar a livello software.

Funzionalità: IOPort come interfaccia web, sonar simulato (rilevazione container e malfunzionamento), aggiornamento display con stato e messaggi.

5.3. Sprint 3: Dispositivi fisici

Goal: integrare, dove disponibili, i dispositivi fisici o i servizi forniti e verificare il comportamento end-to-end.

Funzionalità: cargorobot e servizio DDR disponibile, sonar fisico, marker device fisico, LED fisico.

6. Team di lavoro

NOME E COGNOME	MATRICOLA
Davide Chirichella	0001222371
Gabriele Doti	0001245897
Daniele Maccagnan	0001247340